



University Institute of Engineering
Department of Computer Science & Engineering

Experiment-8

Student Name: Pranav Chauhan
Branch: BE CSE (cyber security)
Semester: 5th
Subject Name: ADBMS

UID: 24BCY70004
Section/Group: 24AIT_KRG-G2
Date of Performance: 07/09/2026
Subject Code: 24CSP-333

- 1. Aim of the practical:** To study and implement **PostgreSQL Views and Materialized Views** for generating complex reports, controlling data access, improving query performance, and managing stored query results.

2. Questions

(A) A company maintains customer information, product information, and sales order information in three separate tables: customer_master, product_catalog, and sales_orders.

The company wants to provide an order report to an external client. The report must contain the **Order ID, Order Date, Customer Name, Product Name, and Final Amount**.

For every order, first calculate the **Gross Amount** using:

$$\text{Gross Amount} = \text{Unit Price} \times \text{Quantity}$$

Then calculate the **Discount Amount** using:

$$\text{Discount Amount} = \text{Gross Amount} \times \text{Discount Percentage} / 100$$

Finally, calculate the **Final Amount** using:

$$\text{Final Amount} = \text{Gross Amount} - \text{Discount Amount}$$

Create a **complex view** using the three tables to generate this report. The view should expose only the required five columns and should not expose the customer's phone/email, product brand/unit price, or the order's quantity/discount percentage.



University Institute of Engineering

Department of Computer Science & Engineering

The company also wants to provide access to an external client. Create a PostgreSQL user named `client_user` and give this user **only SELECT permission on the view**. The client must not have direct access to the three original tables and must not be allowed to modify the view data.

customer_id	full_name	city
C1	Amit Sharma	Delhi
C2	Priya Verma	Mumbai
C3	Ravi Kumar	Bangalore
C4	Neha Singh	Kolkata
C5	Arjun Mehta	Hyderabad

customer_master

product_id	product_name	unit_price
P1	Smartphone X100	25000
P2	Laptop Pro 15	65000
P3	Wireless Earbuds	5000
P4	Smartwatch Fit	30000
P5	Gaming Console	45000

product_catalog

order_id	product_id	quantity	customer_id	discount_percent	order_date
1	P1	2	C1	5	2025-09-01
2	P2	1	C2	10	2025-09-02
3	P3	3	C3	0	2025-09-03
4	P4	1	C4	8	2025-09-04
5	P5	1	C5	12	2025-09-05
6	P1	1	C2	5	2025-09-06
7	P3	2	C1	0	2025-09-07

sales order



University Institute of Engineering

Department of Computer Science & Engineering

Solution:

```
CREATE OR REPLACE VIEW vw_ORDER_PROCESSING
AS
SELECT
    O.order_id,
    O.order_date,
    C.full_name,
    P.product_name,
    (P.unit_price * O.quantity)
    -
    (
        (P.unit_price * O.quantity)
        * O.discount_percent / 100
    ) AS final_amount
FROM customer_master C
JOIN sales_orders O
ON C.customer_id = O.customer_id
JOIN product_catalog P
ON P.product_id = O.product_id;
```

order_id integer	order_date date	full_name character varying (50)	product_name character varying (50)	final_amount numeric
1	2025-09-01	Amit Sharma	Smartphone X100	47500.000000000000000000
2	2025-09-02	Priya Verma	Laptop Pro 15	58500.000000000000000000
3	2025-09-03	Ravi Kumar	Wireless Earbuds	15000.000000000000000000
4	2025-09-04	Neha Singh	Smartwatch Fit	27600.000000000000000000
5	2025-09-05	Arjun Mehta	Gaming Console	39600.000000000000000000
6	2025-09-06	Priya Verma	Smartphone X100	23750.000000000000000000
7	2025-09-07	Amit Sharma	Wireless Earbuds	10000.000000000000000000

(B) A An e-commerce company wants to regularly analyze the performance of its **product categories**. The company stores product, category, customer, and order information in separate tables. To generate the category performance report, the database needs to perform multiple **JOINS**, **GROUP BY**, **aggregate functions**, **DISTINCT counting**, **calculated values**, and **CASE statements**.

Since the same report is generated frequently, executing the complete analytical query every time can increase database workload.

Create a PostgreSQL **Materialized View** named: mv_category_sales



University Institute of Engineering

Department of Computer Science & Engineering

Business Rules

Requirement	Rule
Total Products Sold	SUM(quantity)
Total Orders	COUNT(DISTINCT order_id)
Unique Customers	COUNT(DISTINCT customer_id)
Total Revenue	SUM(quantity * unit_price)
Average Revenue per Order	Total Revenue / Total Orders
Excellent	Revenue ≥ 10,000
Good	Revenue ≥ 6,000 and < 10,000
Average	Revenue ≥ 3,000 and < 6,000
Low	Revenue < 3,000

customer_master

customer_id	customer_name
U1	Aman
U2	Priya
U3	Rahul
U4	Simran

product_master

product_id	product_name	category_id
P101	Laptop	C1
P102	Headphones	C1
P201	Jacket	C2
P202	Shoes	C2
P301	SQL Book	C3

category_master

category_id	category_name
C1	Electronics
C2	Clothing
C3	Books

Solution:

```
create analyze
create or replace materialized view mv_category_sales
SELECT
  c.category_id,
  c.category_name,
  SUM(od.quantity) AS total_products_sold,
  COUNT(DISTINCT od.order_id) AS total_orders,
  COUNT(DISTINCT od.customer_id) AS unique_customers,
  SUM(od.quantity * od.unit_price) AS total_revenue,
  ROUND(
    SUM(od.quantity * od.unit_price)
    / COUNT(DISTINCT od.order_id),
    2
  ) AS avg_revenue_per_order,
  CASE
    WHEN SUM(od.quantity * od.unit_price) >= 10000
    THEN 'Excellent'
    WHEN SUM(od.quantity * od.unit_price) >= 6000
    THEN 'Good'
    WHEN SUM(od.quantity * od.unit_price) >= 3000
    THEN 'Average'
    ELSE 'Low'
  END AS category_performance
FROM category_master c
JOIN product_master p
  ON c.category_id = p.category_id
JOIN order_details od
  ON p.product_id = od.product_id
GROUP BY
  c.category_id,
  c.category_name;
```



University Institute of Engineering
Department of Computer Science & Engineering

3. Learning Outcomes:

- Learned to create and use PostgreSQL Views and Materialized Views.
- Applied joins, aggregate functions, and calculated values in SQL.
- Learned to control user access using permissions and GRANT.
- Understood Materialized View refreshing and stale data.
- Gained practical knowledge of query performance using EXPLAIN ANALYZE.